

# FOODWOK PLATFORM ENGINEERING REPORT

Comprehensive Summary of Development, Unified Auth Architecture Migration, Implemented Policies, and Verification Checks

|                                                                                             |                                            |
|---------------------------------------------------------------------------------------------|--------------------------------------------|
| <b>Project:</b> FOODWOK Web Application & Admin Portal                                      | <b>Date:</b> September 2026                |
| <b>Architecture:</b> Next.js 15, TypeScript, Tailwind, Supabase (Auth + SSR + DB), Paystack | <b>Status:</b> Production Ready & Verified |

## 1. Executive Summary

The FOODWOK web application is a full-stack food ordering platform and administrative management portal engineered for high-performance online food ordering, kitchen dispatching, real-time inventory management, and financial reporting. Designed to support modern Asian-Nigerian culinary commerce, the platform bridges seamless customer ordering with enterprise-grade kitchen operations, unified database architecture, and secure payment processing.

Following a major architectural optimization, the authentication infrastructure was completely migrated from a hybrid Firebase/Supabase system to a unified **Supabase Authentication and SSR middleware framework**. This migration streamlined user session handling, eliminated cross-database synchronization overhead, implemented PKCE-secured Google OAuth, and enforced strict server-side Role-Based Access Control (RBAC).

## 2. Comprehensive Technical & Feature Work Completed

### 2.1 Platform Architecture & Unified Technical Stack

- **Next.js 15 & TypeScript Core:** Configured Next.js 15 App Router architecture utilizing TypeScript and React 19 for full type safety, optimal server/client component boundaries, and high-performance page loading. Zero TypeScript errors across entire source tree.
- **Design System & Styling:** Designed a custom Tailwind CSS styling engine featuring a warm culinary color palette (crimson/amber highlights for storefront, slate/dark theme for kitchen display systems) and responsive layouts.
- **Unified Backend Ecosystem:** Migrated entire backend database and authentication ecosystem to Supabase, eliminating legacy Firebase dependencies. Unified user session state, database profile records (user\_profiles), orders, menu items, and access controls under a single PostgreSQL instance.
- **Modular State Infrastructure:** Built React Context API architecture comprising AuthContext (powered natively by @supabase/supabase-js session listeners), CartContext (persistent cart state/calculations), MenuContext (real-time catalog sync), and OrderContext (order lifecycle management).

### 2.2 Customer Storefront Features

- **Interactive Home & Landing Page:** Constructed responsive hero section with dynamic calls-to-action, category quick-filters, dish highlights, and navigation header with active cart badge indicator.
- **Digital Menu Catalog & Dish Customizer:** Developed comprehensive catalog supporting 23 authentic menu items categorized under Wok & Mains, Small Bites, Noodles, Rice, Soups, Drinks, and Desserts. Included interactive dish modal with spice level selectors, dietary tag badges (Halal, Vegetarian, Vegan, Spicy), and custom instruction fields.
- **Interactive Shopping Cart:** Built dynamic cart overview supporting quantity modifications, item removal, real-time subtotal calculation, and automated delivery fee preview.
- **Checkout & Order Placement:** Engineered multi-step checkout workflow with customer contact collection, zone-based delivery fee calculations, location address verification, and Paystack payment gateway trigger.
- **Real-Time Order Tracking & History:** Implemented order tracking UI rendering visual step-by-step progress bars (Received -> Preparing -> Ready -> Out for Delivery -> Delivered), courier tracking status, ETA estimations, and itemized transaction receipts.

### 2.3 Administrative & Operations Portal

- **Isolated Staff Portal:** Isolated staff and administrator access to a dedicated authentication route (/staff-login), shielding management functions from customer interfaces.

- **Server-Side Guarded KDS:** Built real-time kitchen display dashboard (/admin/kds) protected by server-side @supabase/ssr middleware, supporting active order queue streaming, stage progression controls, and Web Audio API chime notifications for incoming orders.
- **Menu Management Suite:** Created inventory manager (/admin/menu) allowing instant availability toggles (in-stock vs sold-out), price editing, dish description updates, and instant storefront synchronization.
- **Financial Analytics & Accounting Dashboard:** Developed financial analytics interface (/admin/accounting) providing total revenue metrics, daily order volume, average order values, category sales distributions, and revenue trend visual charts.

## 2.4 Unified Authentication Architecture Migration

- **Firebase to Supabase Migration:** Completely replaced Firebase Auth with native Supabase Auth (@supabase/ssr and @supabase/supabase-js), removing legacy Firebase SDK packages (firebase.ts deleted, package.json cleaned up).
- **Google OAuth with PKCE Flow:** Implemented native Google OAuth sign-in (supabase.auth.signInWithOAuth) paired with a server-side PKCE authorization callback route (src/app/auth/callback/route.ts) to exchange auth codes securely and resolve OAuth session race conditions.
- **Supabase SSR Server Middleware:** Rewrote src/middleware.ts using @supabase/ssr (createServerClient) to read server-side cookies, query user\_profiles directly on the server, and enforce strict server-side RBAC (ADMIN or KITCHEN\_STAFF required to access /admin/\*).
- **SMS OTP Authentication:** Refactored phoneLoginSendOTP and phoneLoginVerifyOTP to utilize Supabase native SMS OTP verification (supabase.auth.signInWithOtp / verifyOtp).
- **Single-Source Profile Persistence:** Direct table updates on user\_profiles in Supabase upon profile edits or address changes, removing redundant client-to-server user sync endpoints (/api/users/sync deprecated and removed).

## 2.5 Media Asset Acquisition & Catalog Standardization

- **Physical Menu Digitization:** Digitized 23 physical menu items directly from original restaurant menu photos, setting authentic Nigerian Naira (NGN) pricing, detailed descriptions, and dietary attributes.
- **Media Asset Sourcing:** Acquired, formatted, and integrated high-resolution photographic assets for all 23 menu items across the catalog.
- **Culinary Visual Upgrade:** Replaced 8 specific dish photographs with authentic regional restaurant presentation imagery to match Nigerian culinary styling standards.
- **Framing & Composition Standardization:** Standardized framing and composition across all 23 menu item photos to ensure clean, food-only product displays with zero human background clutter.

## 2.6 Backend Services, Database & Integration Infrastructure

- **Supabase PostgreSQL Database:** Designed PostgreSQL schema with tables for user\_profiles, orders, order\_items, and menu\_items. Configured Row Level Security (RLS) policies and seed scripts for data initialization.
- **Paystack Payment Gateway Integration:** Implemented complete Paystack payment infrastructure including transaction initialization, client checkout redirection, transaction verification, and secure webhook handler with HMAC SHA512 signature validation (x-paystack-signature).
- **Real-Time Synchronization APIs:** Created dedicated sync endpoints (/api/orders-sync, /api/menu-sync, /api/auth/check-email) to maintain real-time state across customer storefront and administrative interfaces.

# 3. Implemented Operational & Technical Policies

To ensure data integrity, customer privacy, system security, and a consistent user experience, the following technical and operational policies were defined and enforced throughout the codebase:

| Policy Name                                     | Scope / Domain    | Policy Description & Enforcement Mechanism                                                                                                                                                                                               |
|-------------------------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Server-Side Supabase SSR Middleware RBAC Policy | Security & Access | All /admin/* routes are protected on the server by @supabase/ssr cookie session verification. Requests lacking ADMIN or KITCHEN_STAFF role in user_profiles are blocked on server and redirected to /staff-login?notice=forbidden_admin. |

|                                                         |                     |                                                                                                                                                                                                |
|---------------------------------------------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Unified Single-Source Auth Policy</b>                | Authentication      | All authentication methods (Email, Google OAuth, SMS OTP) authenticate exclusively through Supabase Auth, storing user profiles directly in user_profiles table without external sync latency. |
| <b>PKCE Google OAuth Callback Policy</b>                | Authentication      | All OAuth authentication attempts must route through /auth/callback to exchange authorization codes for sessions on the server, eliminating client-side session race conditions.               |
| <b>Mandatory Storefront Authentication Policy</b>       | Customer Storefront | Restricts access to checkout, profile management, and order tracking to authenticated users. Unauthenticated visitors are automatically redirected to sign-in with return URL parameters.      |
| <b>Staff Portal Isolation Policy</b>                    | Security & Access   | Separates customer authentication from administrative access. Staff login is strictly routed to /staff-login, guarded by server-side role verification.                                        |
| <b>Unregistered Sign-in Redirection Policy</b>          | Onboarding UX       | Detects when a user attempts sign-in with an email not present in the database, displays an informational notice, and automatically redirects to /auth/signup with pre-filled email.           |
| <b>Phone Format Standardization Policy</b>              | Authentication      | Normalizes local Nigerian phone number inputs (e.g. 080..., 090...) into international standard format (+234...) before initiating SMS OTP verification.                                       |
| <b>Product Photography Framing Policy</b>               | UI / Brand Media    | Mandates that all menu item photos present clean, food-only subjects without human hands, people, or distracting background clutter.                                                           |
| <b>Payment Security &amp; Webhook Validation Policy</b> | Finance & Payments  | Requires HMAC SHA512 signature validation (x-paystack-signature) on all incoming payment webhooks before marking orders as paid in the database.                                               |
| <b>Mandatory Shimmer Loading Skeleton Policy</b>        | UI / Performance    | Enforces that every route in src/app must contain a loading.tsx file rendering animated shimmer skeletons (SkeletonComponents.tsx) to prevent layout shifts.                                   |
| <b>Delivery Fee Calculation Policy</b>                  | Logistics           | Applies standardized zone/distance delivery fee formulas with minimum subtotal logic to ensure transparent shipping cost calculation at checkout.                                              |
| <b>Real-time Kitchen Alerting Policy</b>                | Kitchen Operations  | Triggers mandatory Web Audio API chime notifications on the Kitchen Display System whenever a new order is received in PENDING state.                                                          |

## 4. Comprehensive Quality Checks & Verification Audit

Rigorous quality assurance checks were executed across all platform sub-systems prior to marking the release production-ready. Below is the full audit log of completed verifications:

| Verification Check                                   | Target Module / Route         | Outcome & Verification Details                                                                                                         |
|------------------------------------------------------|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Supabase SSR Middleware RBAC Check</b>            | middleware.ts & /admin/*      | <b>PASSED</b> - Verified unauthenticated and non-admin access attempts are blocked on server and redirected to /staff-login.           |
| <b>Google OAuth &amp; PKCE Callback Verification</b> | /auth/callback & AuthContext  | <b>PASSED</b> - Confirmed PKCE code exchange succeeds cleanly without auth race conditions or stranded session states.                 |
| <b>TypeScript Static Type Safety Check</b>           | Entire Source Tree (src/*)    | <b>PASSED</b> - Zero type compilation errors or broken call signatures detected during tsc --noEmit check.                             |
| <b>Shimmer Skeleton Compliance Check</b>             | All 16 App Routes (src/app/*) | <b>PASSED</b> - Verified loading.tsx containing animated shimmer pulse components exists for every public and admin route.             |
| <b>Native Supabase Auth &amp; Profile Sync Check</b> | AuthContext & user_profiles   | <b>PASSED</b> - Verified Email, Google OAuth, and SMS OTP sign-ins populate user_profiles accurately without secondary sync API calls. |

|                                                    |                                     |                                                                                                                                                               |
|----------------------------------------------------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Paystack HMAC Webhook Verification</b>          | /api/paystack/webhook               | <b>PASSED</b> - Confirmed invalid signature requests are rejected with 401 Unauthorized, and valid paystack webhooks mark orders as PAID.                     |
| <b>Real-Time Menu Toggle Check</b>                 | Admin Menu & Storefront Grid        | <b>PASSED</b> - Confirmed toggling dish availability in Admin Menu instantly updates storefront item cards without manual browser reload.                     |
| <b>Kitchen Audio Chime Notification Check</b>      | Kitchen Display System (/admin/kds) | <b>PASSED</b> - Verified audio chime initializes cleanly via browser Web Audio API upon receipt of new incoming order payload.                                |
| <b>Mobile Responsiveness &amp; UI Layout Check</b> | Storefront & Admin Interfaces       | <b>PASSED</b> - Verified responsive grid breakpoints (1-col mobile, 2-col tablet, 3-col desktop), touch target sizes (44px+), and cart drawer responsiveness. |

## 5. Project Summary & Readiness Declaration

---

The FOODWOK platform development, authentication migration to Supabase SSR, operational policy enforcement, and verification cycles are 100% complete. The platform features robust customer storefront capabilities, server-guarded administrative tools, secure payment handling, unified real-time database synchronization, and complete design policy compliance. The web application is verified ready for production deployment.